This is one of the **biggest sources of confusion** when learning Pandas.

A simple rule is:

Symbol	Meaning	Think Of
[ ]	Selection / Access	"Get me something"
{ }	Dictionary / Mapping	"Define relationships"
( )	Function Call / Execution	"Do something"

---

1. Square Brackets [ ]

Use square brackets when you want to **select, filter, or access data**.

A. Select One Column

df["Salary"]

Think:

> Give me the Salary column.

Output:

```
0    50000
1    60000
2    70000
```

B. Select Multiple Columns

df[["Name", "Salary"]]

Notice:

["Name", "Salary"]

is a list inside brackets.

Output:

Name	Salary
John	50000
Mary	60000

C. Filter Rows

df[df["Salary"] > 50000]

Read it like:

```
df[
   condition
]
```

Example:

```
df[
  df["Salary"] > 50000
]
```

Output:

Name	Salary
Mary	60000
David	70000

D. Access Specific Position

df.iloc[0]

First row.

df.iloc[0:5]

Rows 0 to 4.

Rule for [ ]

Use when:

✅ Selecting columns

✅ Filtering rows

✅ Accessing positions

✅ Accessing dictionary values

---

2. Curly Braces { }

Use curly braces when creating **key-value pairs**.

Think:

Label → Value

A. Create DataFrame

```
df = pd.DataFrame({
  "Name": ["John", "Mary"],
  "Salary": [50000, 60000]
})
```

Here:

```
{
  "Name": [...],
  "Salary": [...]
}
```

means

Column Name -> Data
Column Salary -> Data

B. Rename Columns

```
df.rename(
  columns={
     "Salary": "MonthlySalary"
  }
)
```

Read:

Old Name -> New Name

C. Aggregations

```
df.groupby("Department").agg({
  "Salary": "sum",
  "Age": "mean"
})
```

Read:

Column -> Function

D. Fill Values

```
df.fillna({
  "Salary": 0,
  "Age": 30
})
```

Read:

Column -> Replacement Value

Rule for { }

Use when:

✅ Mapping

✅ Renaming

✅ Aggregation rules

✅ Column definitions

---

3. Parentheses ( )

Use parentheses when **calling a function or method**.

Think:

> Execute something.

A. Read CSV

pd.read_csv("sales.csv")

Function:

read_csv()

Argument:

"sales.csv"

B. Group By

df.groupby("Region")

Call groupby function.

C. Sort Values

```
df.sort_values(
  "Salary",
  ascending=False
)
```

D. Mean

df["Salary"].mean()

Read:

Calculate mean

Rule for ( )

Use when:

✅ Calling functions

✅ Passing parameters

✅ Executing methods

Combining All Three Together

Example:

```
df.groupby("Department").agg({
   "Salary": "sum"
})
```

Let's break it down.

Parentheses

groupby("Department")

Call function.

Curly Braces

```
{
  "Salary": "sum"
}
```

Define mapping.

Parentheses

agg(...)

Call aggregation function.

Most Common Pandas Statement Explained

df[df["Salary"] > 50000]

Step 1:

df["Salary"]

Get Salary column.

Step 2:

df["Salary"] > 50000

Create boolean series.

0 False
1 True
2 True

Step 3:

```
df[
  boolean_series
]
```

Filter rows.

Another Real Example

```
df.rename(
  columns={
     "Salary": "MonthlySalary",
     "Age": "EmployeeAge"
  }
)
```

Parentheses

Function call:

rename(...)

Curly Braces

Mapping:

```
{
 old_column:new_column
}
```

The Golden Memory Trick

Think of a DataFrame as a cupboard.

[ ] = Open a drawer

df["Salary"]

Get something.

{ } = Label stickers

```
{
 "Salary":"MonthlySalary"
}
```

Define relationships.

( ) = Press a button

mean()
sort_values()
groupby()

Execute an action.

Quick Cheat Sheet

```
# Select column
df["Salary"]

# Select multiple columns
df[["Name", "Salary"]]

# Filter rows
df[df["Salary"] > 50000]

# Create DataFrame
pd.DataFrame({
  "Name": ["John"],
  "Salary": [50000]
})

# Rename columns
df.rename(columns={
  "Salary": "MonthlySalary"
})

# Aggregation
df.groupby("Dept").agg({
  "Salary": "sum"
})

# Function calls
df.head()
df.mean()
df.describe()
df.groupby("Dept")
```

Interview Shortcut

When you see Pandas code, ask:

1. **( )** → Is a function being executed?
2. **{ }** → Is a mapping or dictionary being defined?
3. **[ ]** → Is data being selected or filtered?

If you answer those three questions correctly, you'll understand about **90% of Pandas syntax** immediately.